How to Explain MARST to a Friend (Plain English Version)

A casual cheat-sheet for telling someone about the three model variants you built, what they do, and what's actually new.

Start with the problem

> "Imagine the freeway has thousands of little sensors buried in the road that measure how fast cars are going. Now imagine **80% of them just stopped working** — broken wires, dead batteries, network outages, whatever. We need to guess what the broken sensors would have said, using only the 20% that are still reporting."
>
> "That's what my thesis is about. The thing I built is called **MARST**."

Three textbook tricks anyone could think of

> "There are three obvious guesses you could try."

1. LOCF — Last Observation Carried Forward.

> "If sensor 7 last said '58 mph' five minutes ago and now it's broken, just guess 58 again. Works great when traffic is smooth. Falls apart in stop-and-go."

2. HA — Historical Average.

> "If it's Tuesday at 5 PM, look at every previous Tuesday at 5 PM and average what sensor 7 normally read. Works great for predictable rush hours. Falls apart during accidents."

3. KNN — K-Nearest Neighbours.

> "Look at the sensors physically next to sensor 7 that are still reporting. Average them. Works great when traffic is uniform. Falls apart at the edges of traffic jams."
>
> "Each trick works in some situations and fails in others. None of them is invented by us — they've been in textbooks for decades."

What MARST actually is

> "Here's the idea: instead of picking ONE of those three tricks and hoping it works, I built a neural network that decides **at every single sensor, every single moment, which mix of the three tricks to trust** — and then makes a small correction on top.
>
> "So at 5 PM on a smooth highway, my model might say 'I trust the last-value trick 70%, the historical-average trick 25%, the neighbour trick 5%.' At the same instant, during an accident on the next freeway, it might say 'forget the last-value trick, use neighbours 60%, history 30%.'
>
> "And it figures all that out by itself from training data."
>
> "I called it **MARST — Multi-Anchor Residual Spatiotemporal Transformer**. *Anchor* means each of the three classical tricks. *Residual* means the small correction it adds on top. *Spatiotemporal* means it looks at both space (other sensors) and time (the recent past). *Transformer* is the type of neural network."

The three variants I tried

> "I tried three different ways to make it better. The reasoning was: if I'm going to claim this is novel, I should at least try a few variations."

Variant A — Multi-Scale MARST (the 9-anchor version)

> "Instead of 3 tricks, I gave it **9 tricks**. Three versions of LOCF (forgets fast / medium / slow), three versions of HA (hour-scale / day-scale / week-scale), three versions of KNN (1 hop / 2 hops / 3 hops away)."
>
> "Result: didn't really help. The model still picked one favourite per family and ignored the other two. So the extra choices were wasted capacity."

Variant B — Learnable LOCF decay (the per-sensor version)

> "I made one of the dials inside LOCF learnable — let each sensor decide its own forgetting speed."
>
> "Result: the dial barely moved. Every sensor converged to roughly the same value. So per-sensor learnability isn't really useful here."

Variant C — Cross-domain test (electricity)

> "I took the same model with the same settings and ran it on a totally different problem — predicting electricity consumption from sparse household meters."
>
> "Result: **it worked.** Beat the simple baseline by 25%. Same model, no retuning. That tells us the architecture isn't traffic-specific — it's a general method that happens to be applied to traffic."

The actual winner — load-balancing fix

> "While doing Variant A, I noticed the gate was being **lazy**. It would lock onto one favourite trick per dataset and basically ignore the other two. This is a well-known problem in machine learning called 'expert collapse' — when you give a model a choice, it tends to overcommit to one option."
>
> "There's a known fix from Google's Switch Transformer paper (2021): add a small extra penalty during training that forces the model to keep all options on the table."
>
> "I applied that fix. The model now uses all three tricks more evenly **and wins on all four of the standard traffic datasets** against 18 other published methods — including the 2024 state of the art. The biggest improvement is 17% MAE reduction on PEMS08 over the strongest competitor. All four wins are statistically significant (Wilcoxon test, $p<0.001$)."
>
> "That's the version I'm putting in my thesis."

How to explain what's actually 'novel' (honestly)

> "Honestly? I didn't invent anything new. Every piece is from somewhere else: - The transformer is from 2017. - The three tricks are textbook. - The mixing idea is from old hybrid models like ES-RNN (which won M4 in 2018). - The load-balancing fix is from Switch Transformer (2021).
>
> "But **the specific combination** — using textbook tricks as inputs to a strictly-causal transformer with a learned per-position mixture, applied to traffic imputation — hadn't been published before. I checked. There's a 2024 survey covering 11 methods in this space and nobody does this exact combination.
>
> "Plus I added one engineering thing that's actually mine: I noticed the gate was collapsing, diagnosed it using the expert-collapse literature, applied the load-balancing fix, and showed it improves results. That's a small but legitimate contribution.

> "And the model wins on **all four** of the standard benchmarks against 18 other published methods — including the 2024 state of the art — at a statistically significant margin. Plus it works on electricity too.
>
> "So it's not Einstein-level novel. It's careful engineering with a small clever twist, evaluated rigorously. That's what an MS thesis is supposed to be."

If your friend asks 'why is this useful?'

> "Real traffic monitoring systems are missing 20-30% of their data on any given day. Without good imputation, the systems downstream — adaptive signal timing, traffic-aware routing, incident detection — all degrade.
>
> "If my model is 14% more accurate than the best existing method on flow datasets, that translates to real improvements in those downstream applications. Plus it runs in real time (it's strictly causal — no peeking at the future), so it can be deployed as a live system. Most of the published methods can't claim that."

If your friend asks 'how did you test it?'

> "Four standard traffic datasets that every paper in this space uses: PEMS-BAY (Bay Area freeways), METR-LA (LA urban roads), PEMS04 and PEMS08 (California freeway flow counts). I ran 18 baseline methods plus mine, three random seeds each, five different test masks. So every reported number is averaged over 15 runs. I used the Wilcoxon signed-rank test with Holm correction to confirm the wins are statistically significant.
>
> "I also did a leak audit on every model — corrupting the held-out truth and verifying no model accidentally sees it. All 19 models passed. Mine also passes a causality test where future inputs can't affect past predictions."

If your friend asks 'is it deep learning magic or what?'

> "It's a smart combination of three boring ideas: 1. A textbook prediction trick gives the model a sensible starting guess. 2. A transformer learns when to trust each trick. 3. A small neural correction patches the remaining errors.
>
> "Most deep learning papers throw the textbook tricks away and ask the model to rediscover them. I figured: why waste the model's capacity? Just hand it the textbook tricks as inputs, and let it learn the corrections.

> "Turns out this is a known good idea — it's how ES-RNN won the M4 forecasting competition in 2018. I'm just the first to apply that strategy to streaming traffic imputation."

The 30-second version

If your friend has no patience:

> "Traffic sensors break a lot. I built a model that fills in the missing readings by mixing three textbook tricks and adding a small correction. It wins on **all four** standard benchmarks against 18 other methods (including the 2024 state of the art) at a statistically significant margin, and it also works on electricity data without changes. The clever bit is that I noticed the gate was lazy and added a known fix to make it use all three tricks evenly. Nothing I built is fundamentally new — I just combined good ideas carefully and tested rigorously. That's what an MS thesis is supposed to be."

That's it. You're done.